

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет программной инженерии и компьютерной техники

Направление подготовки: 09.03.04 – Системное и прикладное
программное обеспечение
Дисциплина «Вычислительная математика»

Лабораторная работа №2

Вариант 7

Выполнила

Матвеева Полина Павловна

Проверила

Наумова Надежда Александровна

Санкт-Петербург, 2026

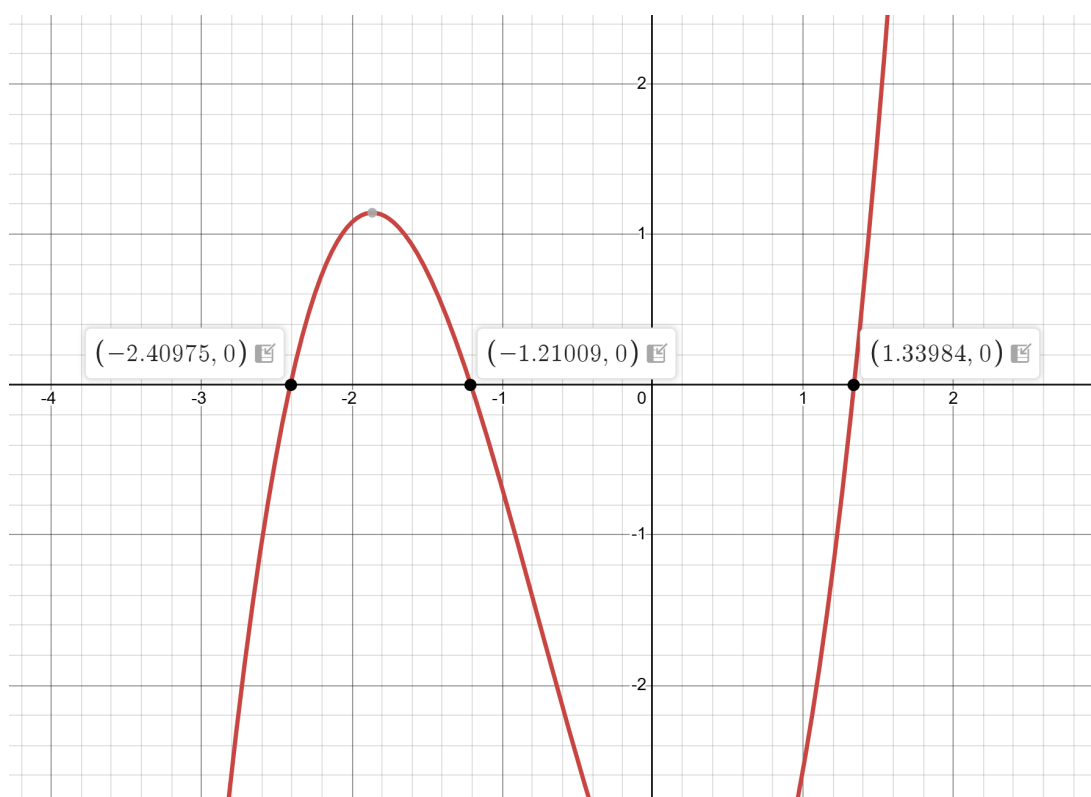
Цель

Изучить численные методы решения нелинейных уравнений и их систем, найти корни заданного нелинейного уравнения/системы нелинейных уравнений, выполнить программную реализацию методов.

1. Вычислительная реализация задачи

1. Решение нелинейного уравнения

1. $x^3 + 2.28x^2 - 1.934x - 3.907$



2. Для определения интервалов изоляции корней данного уравнения воспользуемся методом интервалов знакопеременности.

Приближенные значения корней:

$$x \approx -2.41, \quad x \approx -1.21, \quad x \approx 1.34$$

Разобьем ось x на интервалы:

$$(-\infty; -2.41), (-2.41; -1.21), (-1.21; 1.34), (1.34; +\infty)$$

Определим графически знак функции на каждом из интервалов:

$(-\infty; -2.41)$	$(-2.41; -1.21)$	$(-1.21; 1.34)$	$(1.34; +\infty)$
-	+	-	+

Таким образом получаем интервалы изоляции корней:

$$(-3; -2), \quad (-2; -1), \quad (1; 2)$$

3. Уточним корни до двух знаков после запятой

$$x_1 \approx -2.41$$

$$x_2 \approx -1.21$$

$$x_3 \approx 1.34$$

4. Крайний правый корень — метод половинного деления

№	a	b	x	$f(a)$	$f(b)$	$f(x)$	$ a - b $
1	1	2	1.5	-2.561	9.345	1.697	1
2	1	1.5	1.25	-2.561	1.697	-0.809	0.5
3	1.25	1.5	1.375	-0.809	1.697	0.344	0.25
4	1.25	1.375	1.3125	-0.809	0.344	-0.257	0.125
5	1.3125	1.375	1.34375	-0.257	0.344	0.037	0.0625
6	1.3125	1.34375	1.328125	-0.257	0.037	-0.111	0.03125
7	1.328125	1.34375	1.33594	-0.111	0.037	-0.037	0.0156

5. Крайний левый корень — метод простой итерации

№	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	-2.800	-2.543	-0.690	0.257
2	-2.543	-2.474	-0.310	0.069
3	-2.474	-2.443	-0.155	0.031
4	-2.443	-2.428	-0.082	0.016
5	-2.428	-2.419	-0.044	0.008
6	-2.419	-2.415	-0.024	0.004
7	-2.415	-2.413	-0.013	0.002

6. Центральный корень — метод Ньютона

Производная функции:

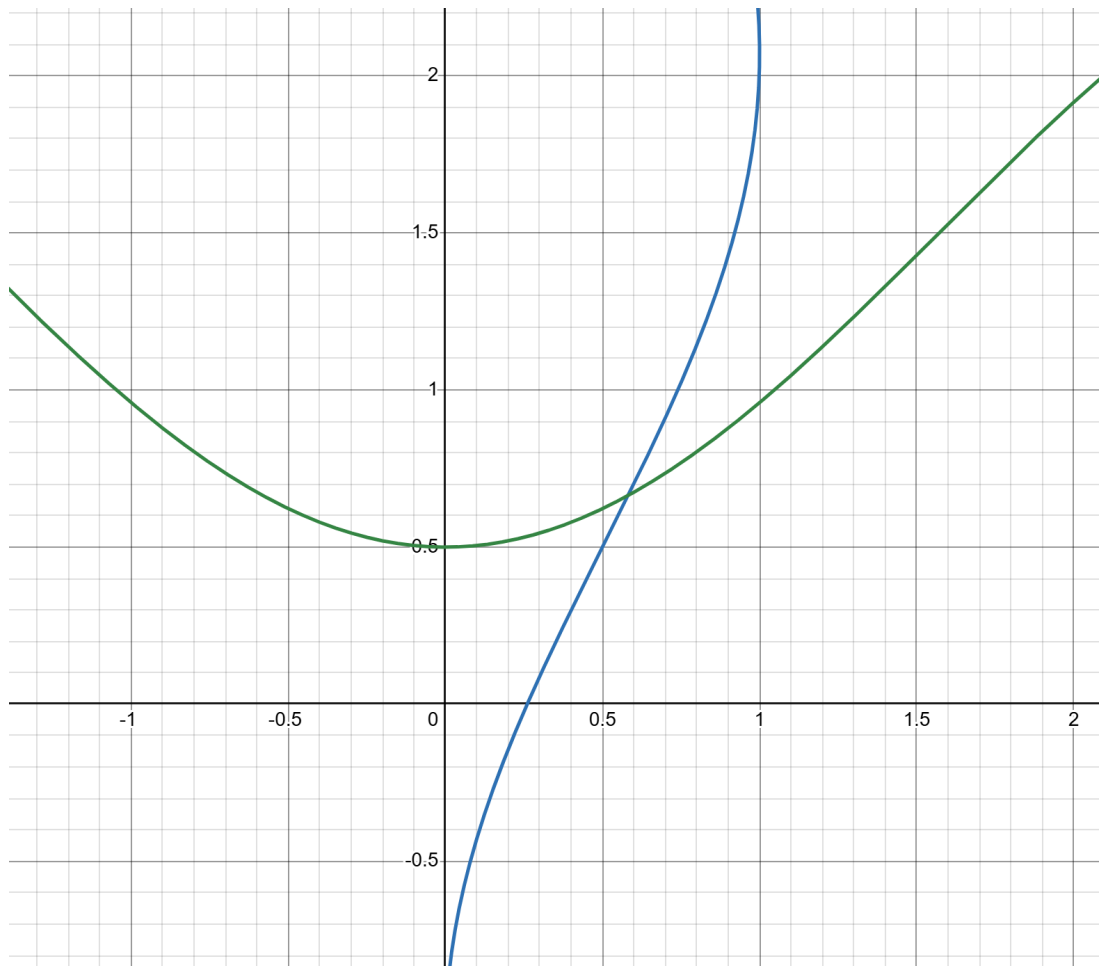
$$f'(x) = 3x^2 + 4.56x - 1.934$$

№	x_k	$f(x_k)$	$f'(x_k)$	x_{k+1}	$ x_{k+1} - x_k $
1	-1.500	0.749	-2.024	-1.130	0.370
2	-1.130	-0.253	-3.256	-1.208	0.078
3	-1.208	-0.007	-3.065	-1.210	0.002

2. Решение системы нелинейных уравнений

1.

$$\begin{cases} 2x - \sin(y - 0.5) = 1 \\ y + \cos x = 1.5 \end{cases}$$



2. Анализируя графики функций, можно определить область пересечения кривых. Точка пересечения находится приблизительно в области:

$$0.5 \leq x \leq 0.7 \text{ и } 0.6 \leq y \leq 0.7.$$

Выберем начальное приближение:

$$x_0 = 0.6, \quad y_0 = 0.7$$

3. Проверка условия сходимости

Представим систему в виде:

$$\begin{cases} x = \phi_1(x, y) = \frac{1 + \sin(y - 0.5)}{2} \\ y = \phi_2(x, y) = 1.5 - \cos x \end{cases}$$

Найдём частные производные:

$$\frac{\partial \phi_1}{\partial x} = 0, \quad \frac{\partial \phi_1}{\partial y} = \frac{1}{2} \cos(y - 0.5)$$

$$\frac{\partial \phi_2}{\partial x} = \sin x, \quad \frac{\partial \phi_2}{\partial y} = 0$$

Проверим условие сходимости в окрестности корня ($x \approx 0.58$, $y \approx 0.66$):

$$|0| + \left| \frac{1}{2} \cos(0.66 - 0.5) \right| \approx 0.493 < 1$$

$$|\sin(0.58)| + |0| \approx 0.551 < 1$$

Поскольку суммы меньше 1, условие сходимости выполняется и метод простой итерации сходится.

4. Решение методом простой итерации

k	x_k	y_k	Δx	Δy
0	0.600	0.700	-	-
1	0.599	0.674	0.001	0.026
2	0.587	0.666	0.012	0.008
3	0.583	0.665	0.004	0.001
4	0.582	0.664	0.001	0.001

2. Программная реализация

В данном разделе представлена программная реализация изученных методов. Алгоритмы визуализированы в виде блок-схем, а логика работы отражена в исходном коде программы.

2.1. Методы решения нелинейных уравнений

2.1.1 Метод секущих

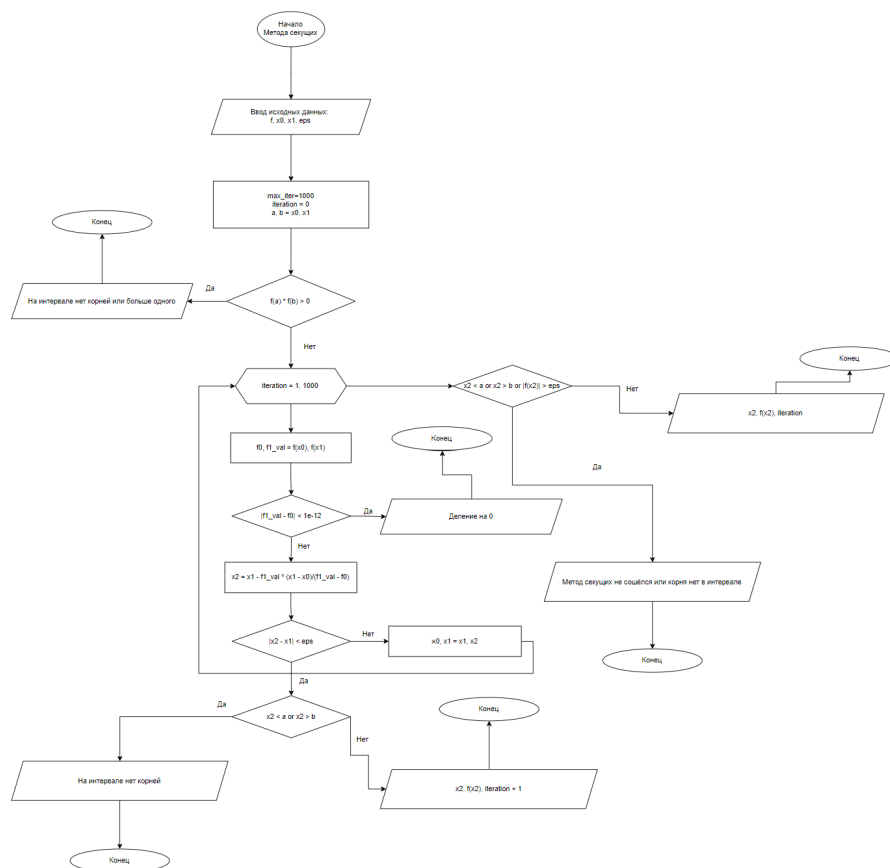


Рисунок 3. Блок-схема метода секущих

```

def secant_method(f, x0, x1, eps, max_iter=1000): 1 usage
    iteration = 0
    a, b = x0, x1
    if f(a) * f(b) > 0:
        raise ValueError("На интервале нет корней или корней больше одного")
    while iteration < max_iter:
        f0, f1_val = f(x0), f(x1)
        if abs(f1_val - f0) < 1e-12:
            raise ValueError("Деление на ноль")
        x2 = x1 - f1_val * (x1 - x0) / (f1_val - f0)
        if abs(x2 - x1) < eps:
            if x2 < a or x2 > b:
                raise ValueError("На интервале нет корней или корней больше одного")
            return x2, f(x2), iteration + 1
        x0, x1 = x1, x2
        iteration += 1
    if x2 < a or x2 > b or abs(f(x2)) > eps:
        raise ValueError("Метод секущих не сошёлся или корня нет в интервале")
    return x2, f(x2), iteration

```

Рисунок 4. Программная реализация метода секущих

2.1.2 Метод хорд

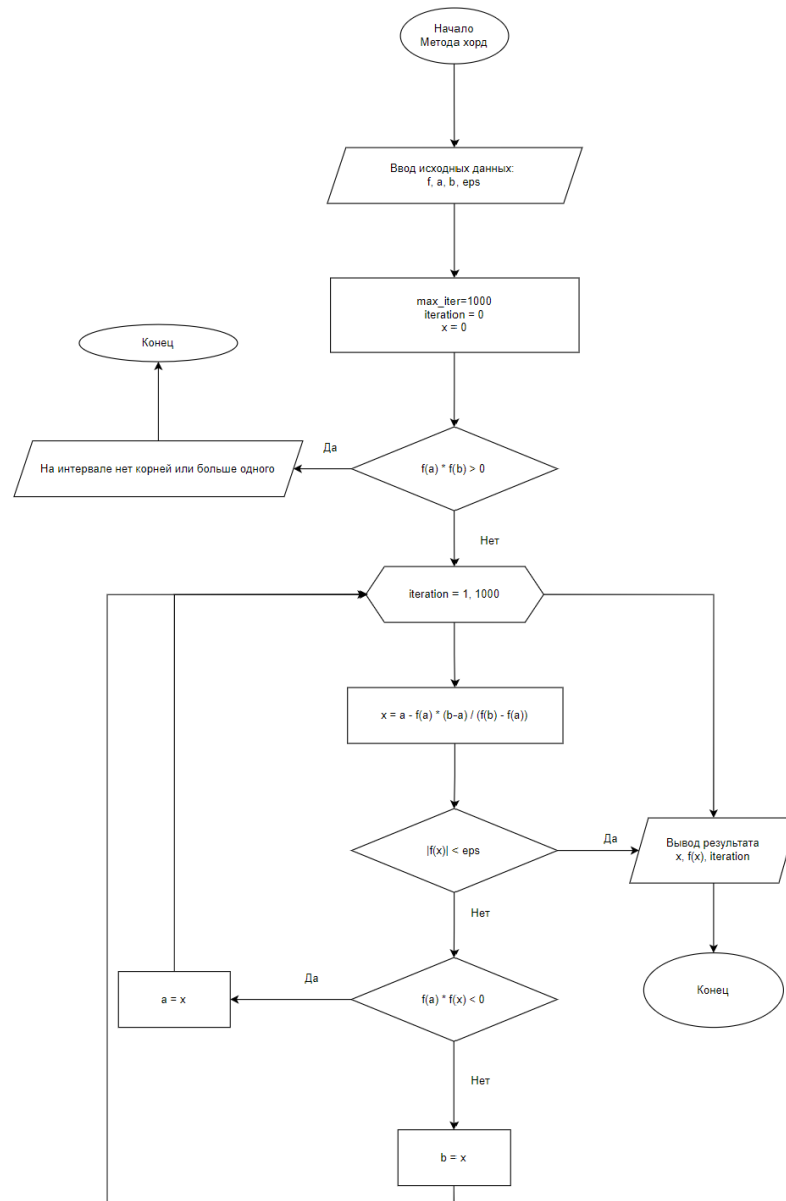


Рисунок 5. Блок-схема метода хорд

```

def chord_method(f, a, b, eps, max_iter=1000): 1 usage
    iteration = 0
    x = 0
    if f(a) * f(b) > 0:
        raise ValueError("На интервале нет корней или корней больше одного")
    while iteration < max_iter:
        x = a - f(a) * (b - a) / (f(b) - f(a))
        if abs(f(x)) < eps:
            return x, f(x), iteration
        if f(a) * f(x) < 0:
            b = x
        else:
            a = x
        iteration += 1
    return x, f(x), iteration

```

Рисунок 6. Программная реализация метода хорд

2.1.3 Метод простых итераций (для уравнения)

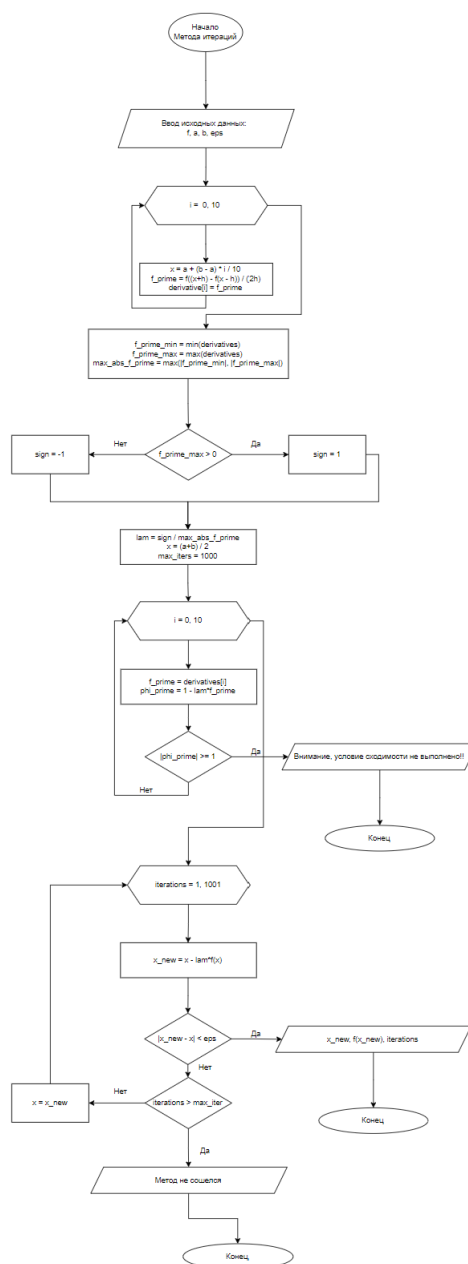


Рисунок 7. Блок-схема метода простых итераций

```

def simple_iteration(f, a, b, eps): 1 usage
    steps = 10
    derivatives = [get_derivative(f, a + (b - a) * i / steps) for i in range(steps + 1)]
    f_prime_min = min(derivatives)
    f_prime_max = max(derivatives)
    max_abs_f_prime = max(abs(f_prime_min), abs(f_prime_max))
    sign = 1 if f_prime_max > 0 else -1
    lam = sign / max_abs_f_prime
    x = (a + b) / 2
    iterations = 0
    max_iters = 1000
    for f_prime in derivatives:
        phi_prime = 1 - lam * f_prime
        if abs(phi_prime) >= 1:
            raise ValueError(f"Внимание, условие сходимости не выполнено!! |phi'| = {abs(phi_prime):.4f}")
    while True:
        x_new = x - lam * f(x)
        iterations += 1
        if abs(x_new - x) < eps:
            return x_new, f(x_new), iterations
        if iterations > max_iters:
            raise ValueError(f"Метод не сошелся за 1000 итераций. "
                               f"Возможно, не выполнено условие |phi'(x)| < 1 на этом отрезке.")
        x = x_new

```

Рисунок 8. Программная реализация метода простых итераций

2.2. Метод простых итераций для систем уравнений

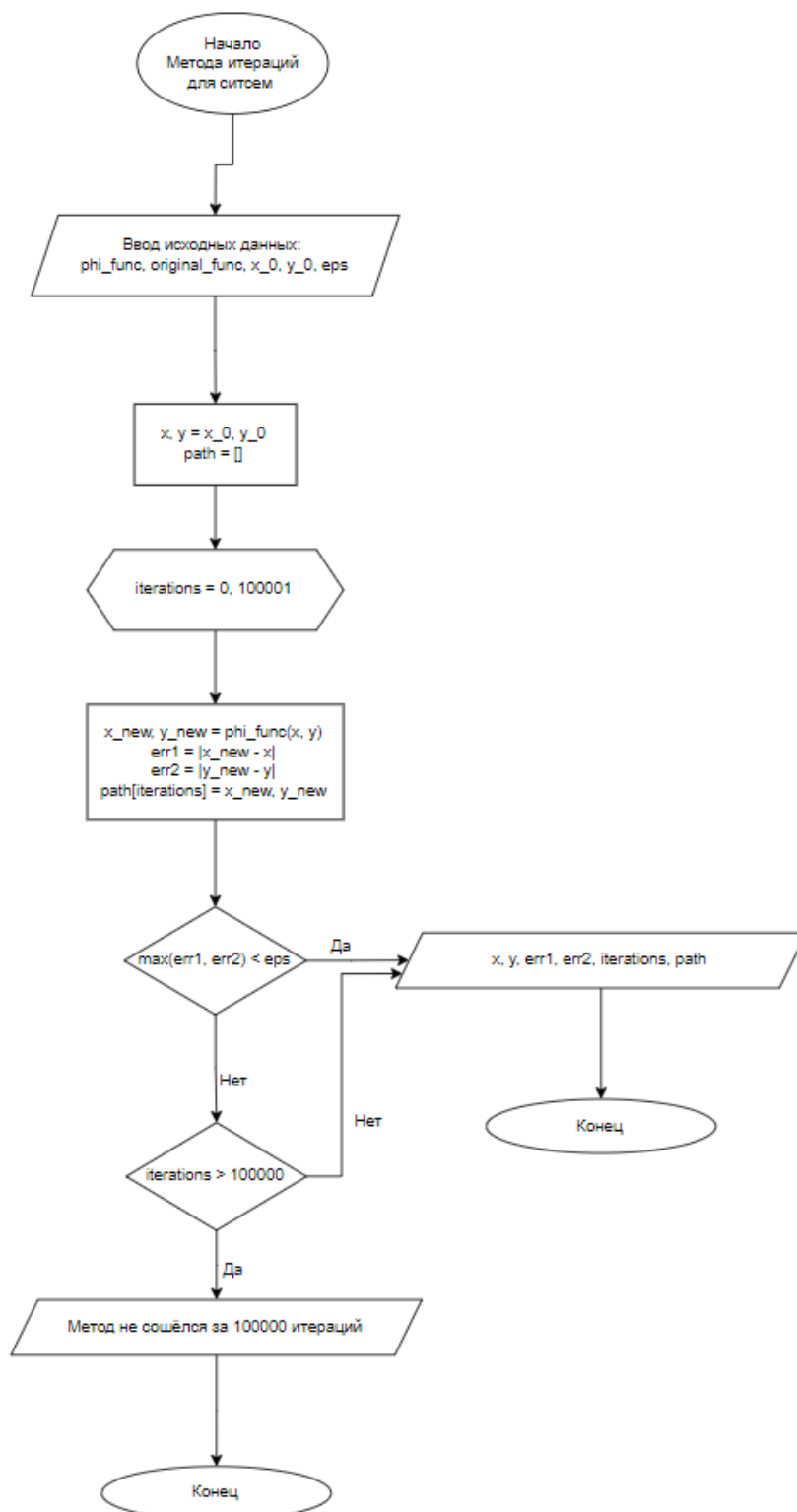


Рисунок 9. Блок-схема МПИ для систем уравнений

```

def simple_iteration_system(sys_name, x_0, y_0, eps): 1 usage
    systems = {
        'sys1': (sys1_phi, sys1_original),
        'sys2': (sys2_phi, sys2_original),
    }
    phi_func, original_func = systems[sys_name]
    is_convergent, norm = check_convergence(phi_func, x_0, y_0)
    if not is_convergent:
        raise ValueError(f"Норма матрицы Якобиана {norm} > 1")
    x, y = x_0, y_0
    iterations = 0
    path = []
    while True:
        x_new, y_new = phi_func(x, y)
        iterations += 1
        err1 = abs(x_new - x)
        err2 = abs(y_new - y)
        path.append((x_new, y_new))
        x, y = x_new, y_new
        if max(err1, err2) < eps:
            break
        if iterations > 100000:
            raise ValueError("Метод не сошёлся за 100000 итераций")
    return x, y, err1, err2, iterations, path

```

Рисунок 10. Программная реализация решения системы уравнений

3. Результаты работы программы

В данном разделе представлены скриншоты ответов на страницах сайта, содержащие графические решения, количество итераций и найденные значения корней.

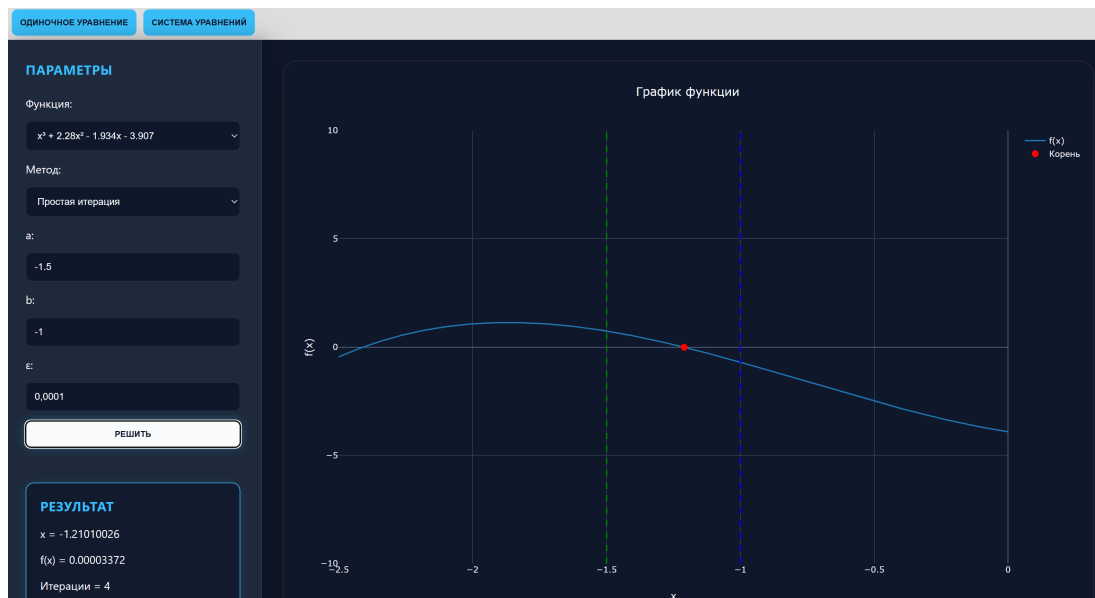


Рисунок 11. Результат решения нелинейного уравнения

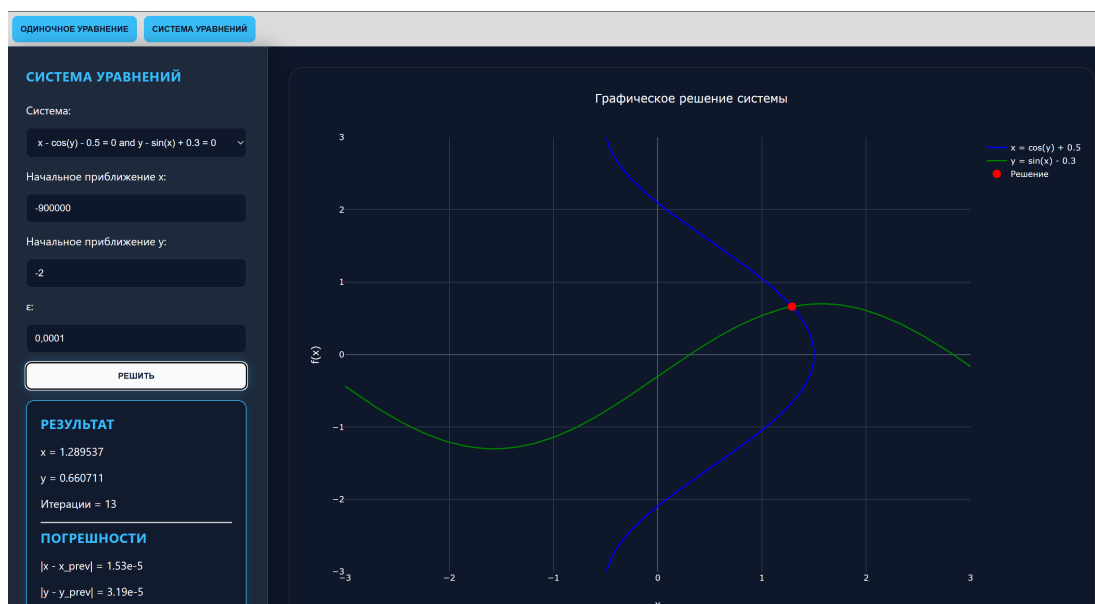


Рисунок 12. Результат решения системы уравнений

Вывод

В ходе выполнения лабораторной работы были изучены и программно реализованы численные методы решения нелинейных уравнений и их систем.

При реализации методов для одиночных уравнений (секущих, хорд, простых итераций) была проведена предварительная изоляция корней, что позволило корректно задать начальные приближения. Для

системы уравнений было успешно проверено условие сходимости в выбранной области, что подтвердило корректность работы метода простых итераций. Сравнение результатов ручного счета и программного вывода показало их сходимость с заданной точностью ϵ , что говорит о правильности выбранных алгоритмов и их реализации.